ECU
EGYPTIAN CHINESE UNIVERSITY
الجامعة المصرية الصينية
EDUCATION NEW ERA

Student Grade System

Name:	ID:
Mohamed Monged Mostafa	**192300248**
Martin Milad	**192300344**
Nicole Aziz	**192300428**
Mina Ehab fahem	**192300365**
Seif El Den Waleed Ayed	**192300230**

Under the supervision of

Dr/Cherry Aly

TA/Hussen Mohamed

TA/Mohamed Hany

Evaluation Criteria

Criteria	Excellent (Full Marks)	Very Good	Good	Fair	Poor	Marks
Encapsulation (2)	2	1.5	1	0.5	0	_____
Inheritance (1.5)	1.5	1	0.5	0.25	0	_____
Abstraction (1.5)	1.5	1	0.5	0.25	0	_____
Polymorphism (1.5)	1.5	1	0.5	0.25	0	_____
Code Quality (1)	1	0.75	0.5	0.25	0	_____
Functionality (1.5)	1.5	1	0.5	0.25	0	_____
Presentation (1)	1	0.75	0.5	0.25	0	_____

1. Introduction

The **Student Grade System** is an Object-Oriented Programming (OOP) project developed using **C++** and the **Qt Framework**. The system provides a graphical user interface (GUI) to manage students, professors, and academic grades in an organized and extensible way. It is designed as an educational project to demonstrate core OOP principles such as encapsulation, inheritance, abstraction, and modular design.

2. Project Objectives

- Apply **OOP concepts** in a real-world academic system
- Practice **Qt GUI development** using dialogs and widgets
- Manage **students and professors** data efficiently
- Provide a clear separation between **logic and interface**
- Build a scalable foundation for future extensions (subjects, GPA, reports, etc.)

3. Technologies Used

- **Programming Language:** C++
- **Framework:** Qt 6
- **IDE:** Qt Creator
- **Build System:** CMake / qmake (Qt default)
- **Platform:** Windows

4. Project Structure

```
Student Grade System
|
├── Source Files (.cpp)
|   ├── main.cpp
|   ├── MainWindow.cpp          (GUI)
|   ├── Student.cpp
|   ├── Professor.cpp
|   ├── Course.cpp
|   ├── Grade.cpp
|   ├── GradeManager.cpp
|   ├── StudentDialog.cpp       (GUI)
|   ├── ProfessorDialog.cpp     (GUI)
|   ├── CourseDialog.cpp        (GUI)
|   └── GradeDialog.cpp         (GUI)
|
├── Header Files (.h)
|   ├── MainWindow.h            (GUI)
|   ├── Person.h
|   ├── Student.h
|   ├── Professor.h
|   ├── Course.h
|   ├── Grade.h
|   ├── GradeManager.h
|   ├── DataRepository.h
|   ├── StudentDialog.h         (GUI)
|   ├── ProfessorDialog.h       (GUI)
|   ├── CourseDialog.h          (GUI)
|   └── GradeDialog.h           (GUI)
|
├── Resources
|   └── Qt styles
|
└── Build & Configuration
    ├── .pro / CMakeLists.txt
    └── Generated build files
```

5. System Architecture (Layered)

1. Presentation Layer (GUI – Qt)

- `MainWindow`
- Dialog classes (Student, Course, Grade, Professor)
- Handles user input and data display only
- **No business logic**

2. Management / Control Layer

- `GradeManager`
- `DataRepository`
- Controls system workflow (enrollment, grading, GPA, persistence)
- Acts as the central coordinator

3. Core Logic Layer

- `Person`
- `Student`
- `Professor`
- `Course` (Core, Elective, Lab)
- `Grade`
- Contains all academic rules and OOP logic

6. OOP Concepts Used

6.1 Encapsulation

- Data members are declared as **private**
- Access is provided through **public getters and setters**

6.2 Abstraction

- Classes represent real-world entities such as *Student* and *Professor*
- Implementation details are hidden from the UI

6.3 Inheritance

- Common attributes can be shared through base classes (e.g., Person concept)

6.4 Polymorphism

- Functions can be overridden to customize behavior for different entities

7. Class Documentation

7.1 Student Class

Header: `Student.h`
Source: `Student.cpp`

Responsibilities:

- Store student personal and academic information
- Manage grades and calculate academic results

Main Attributes:

- Student ID
- Name
- Major
- Grades

Key Methods:

- Constructors
- Getters and setters
- GPA and grade calculation methods

OOP Concepts Used:

- **Encapsulation:** Student data members are private and accessed only through public getter and setter methods.
- **Inheritance:** The Student class inherits common attributes and behavior from the abstract Person class.
- **Abstraction:** Student provides concrete implementations of abstract methods defined in Person.
- **Polymorphism**: Student objects can be accessed through Person* pointers while executing overridden methods at runtime.

7.2 Professor Class

Header: `Professor.h`
Source: `Professor.cpp`

Responsibilities:

- Store professor information
- Manage courses assigned to the professor

Main Attributes:

- Professor ID
- Name
- Department

Key Methods:

- Constructors
- Getters and setters
- Course assignment methods

OOP Concepts Used:

- **Encapsulation:** Professor attributes are hidden and modified only through controlled public methods.
- **Inheritance:** The Professor class extends the Person base class to reuse common personal information.
- **Abstraction:** Professor implements abstract functions declared in the Person class.
- **Polymorphism:** A Person* can reference a Professor object and call the overridden methods dynamically.

7.3 Course Class

Header: `Course.h`
Source: `Course.cpp`

Responsibilities:

- Store course details
- Handle student enrollment
- Calculate final grades

Main Attributes:

- Course ID
- Course Name
- Credits
- Enrolled Students

Key Methods:

- Enroll and remove students
- Grade calculation methods

OOP Concepts Used:

- **Encapsulation:** Course data such as enrolled students and credits is protected and accessed through class methods.
- **Abstraction:** Course is an abstract class that defines common course behavior without implementation details.
- **Inheritance:** CoreCourse, ElectiveCourse, and LabCourse inherit from Course to specialize behavior.
- **Polymorphism:** Different course types override grade calculation methods, executed through base class pointers.

7.4 Grade Class

Header: `Grade.h`
Source: `Grade.cpp`

Responsibilities:

- Store and manage student grades
- Calculate total and letter grades

Main Attributes:

- Student ID
- Course ID

- Scores

Key Methods:

- Set and update scores
- Calculate total and letter grade

OOP Concepts Used

- **Encapsulation:** Grade values are private and updated only through controlled setter methods.
- **Abstraction:** Grade hides the internal calculation logic behind public methods such as total and letter grade calculation.

7.5 GradeManager Class

Header: `GradeManager.h`
Source: `GradeManager.cpp`

Responsibilities:

- Control system logic
- Manage students, courses, professors, and grades
- Handle data storage and retrieval

Main Attributes:

- Collections of students, courses, professors, and grades

Key Methods:

- Add, remove, and search operations
- Enrollment and assignment functions

OOP Concepts Used:

- **Encapsulation:** Internal data collections are private and accessed only through management functions.
- **Abstraction:** GradeManager exposes high-level system operations without revealing internal implementation details.
- **Singleton Design Pattern:** A single instance of GradeManager is enforced to centralize system control.

7.6 StudentDialog Class

Header: `StudentDialog.h`
Source: `StudentDialog.cpp`

Responsibilities:

- Provide GUI for adding and editing student data
- Display student information

Qt Components Used:

- `QDialog`
- `QLineEdit`
- `QPushButton`

OOP Concepts Used

- **Encapsulation:** UI components are private and manipulated through dialog functions.
- **Abstraction:** The dialog hides UI handling details from the system logic.
- **Inheritance:** StudentDialog inherits from QDialog to reuse dialog behavior.
- **Polymorphism:** Qt's signal and slot mechanism enables dynamic response to user actions.

7.7 MainWindow Class

Header: `MainWindow.h`
Source: `MainWindow.cpp`

Responsibilities:

- Control the main user interface
- Manage system navigation and actions

Qt Components Used:

- `QMainWindow`
- `QTabWidget`
- `QTableWidget`

OOP Concepts Used

- **Encapsulation:** UI elements and actions are contained within the MainWindow class.
- **Abstraction:** MainWindow provides a high-level interface without exposing internal widget logic.
- **Inheritance:** MainWindow extends QMainWindow to reuse core window functionality.
- **Polymorphism:** Qt event handling allows overridden behavior for user interactions at runtime.

8. User Interface Design

- Dialog-based UI
- Simple and clean layout
- Input validation through Qt widgets

9. Functional Features

- Add new students
- Add professors
- Display stored information
- Basic grade management

10. Error Handling

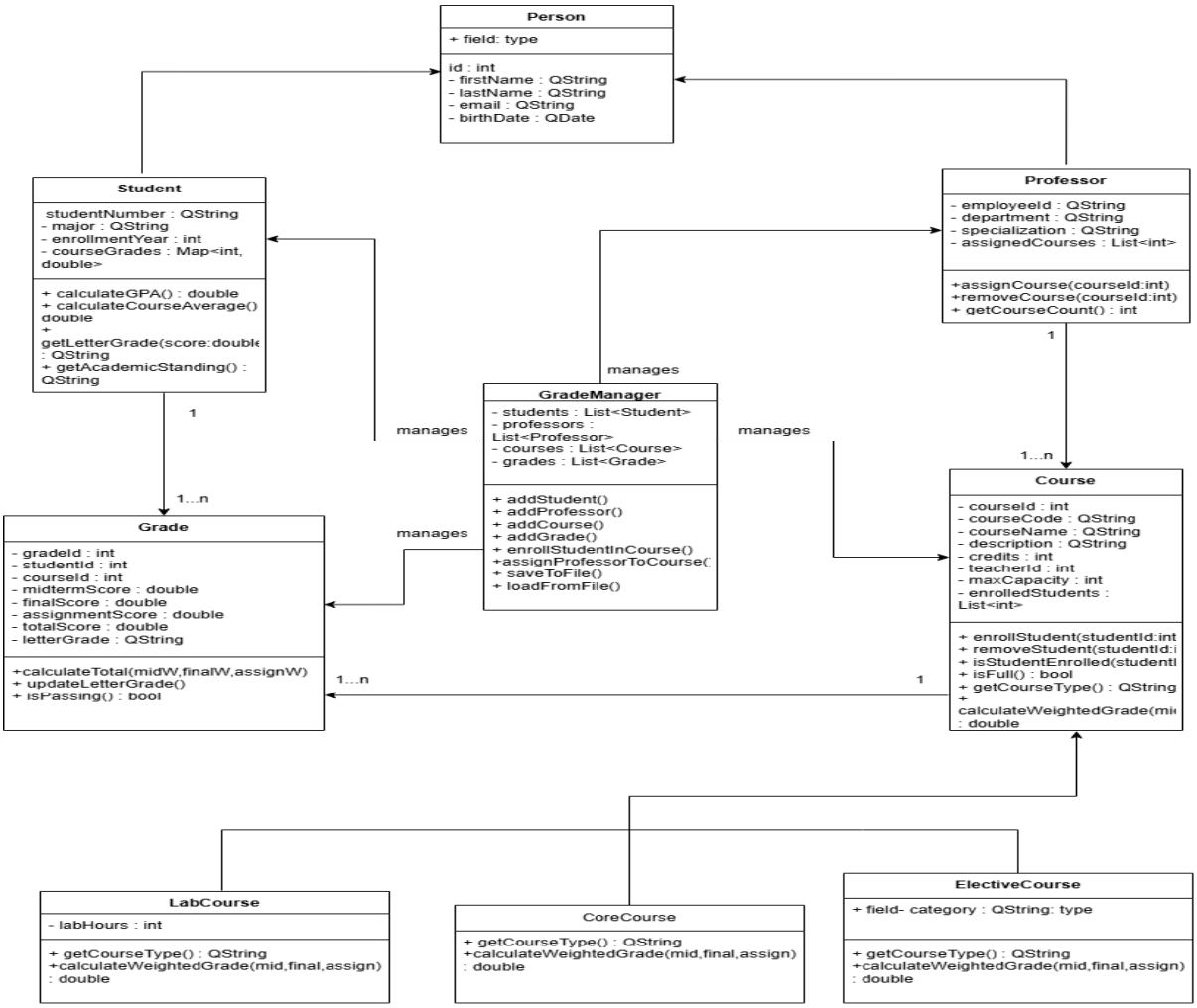
- Input validation using Qt
- Safe object initialization
- Controlled access to class members

11. Future Enhancements

- Subject management module

- GPA calculation
- Database integration (SQLite)
- User authentication
- Report generation (PDF)

12. Class Diagram



13. Conclusion

The **Student Grade System** project successfully demonstrates the practical application of **Object-Oriented Programming** using **C++ and Qt**. It provides a solid foundation for managing academic data while maintaining clean architecture and extensibility. The project can be easily expanded to meet more advanced academic system requirements.